T.Y.B.C.A.

JAVA PROGRAMMING

PRACTICAL RECORD

Course Code	CA-315
Course Category	DSC-19
Course Title	Practical based on JAVA Programming
Course Type	Practical
Contact Hours	60 (4/week)
Credits	02
College Assessment (CA)	20 Marks
University Assessment (UA)	30 Marks

Developed by

Assistant Professor Tahir Mansuri

Institute of Management Research & Development (IMRD), Shahada

Course Objectives

- To develop practical skills in core Java programming and problem-solving.
- To apply object-oriented programming principles such as classes, objects, inheritance, polymorphism and interfaces.
- To use exception handling and perform basic file/database-related programming tasks.
- To develop simple real-world Java applications, including graphical user interfaces and database operations.

Course Outcomes

- Write, compile and execute Java programs using core language constructs.
- Design programs using classes, objects, constructors, inheritance, interfaces and polymorphism.
- Build basic GUI applications using AWT/Swing components and event handling.
- Connect Java applications to a database and perform common data operations using JDBC.

List of Practicals

1. Display Student Information
2. Arithmetic Operations Using switch Case
3. Find the Largest Element in an Array
4. Use Command-Line Arguments
5. Accept User Input Using Scanner
6. Demonstrate Constructors and Methods
7. Demonstrate Method Overloading
8. Demonstrate Inheritance
9. Demonstrate Method Overriding
10. Interface for Shape Area Calculation
11. String Manipulation Operations
12. Exception Handling Using try-catch-finally
13. Create a Simple Calculator Using Swing
14. Create a Simple Login Form Using Swing
15. Demonstrate Database Connectivity Using JDBC
16. Insert, Update and Delete Database Records

General Instructions

- Save each program using the same name as its public class. If a class is not public, save it under a suitable .java filename.
- Compile a console program with: javac FileName.java. Run it with: java ClassName.
- For Swing practicals, run the program in a graphical desktop session; a GUI window will open.
- For JDBC practicals, create the sample database/table, install MySQL Connector/J and update the connection credentials before execution.

PRACTICAL 1: Display Student Information

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To write a Java program that displays basic student information using output statements.

Theory / Introduction

Java is a class-based, object-oriented programming language. A Java application generally contains a class and a main() method, which acts as the entry point when the program runs.

System.out.println() displays text and moves the cursor to the next line. System.out.print() displays text without moving to a new line. Text enclosed in double quotation marks is a String literal.

Important Points

- Java source files are saved with the .java extension.
- The public class name must match the source filename.
- Java statements generally end with a semicolon.

Program

```
class StudentInfo {
    public static void main(String[] args) {
        System.out.println("Student Name: James");
        System.out.println("Class: TYBCA");
        System.out.println("Course: Java Programming");
        System.out.println("College: IMRD, Shahada");
    }
}
```

Sample Output

```
Student Name: James
Class: TYBCA
Course: Java Programming
College: IMRD, Shahada
```

Explanation

The main() method runs first. Each println() statement prints one line of student information.

Result

Thus, the program for display student information was written and its expected output was studied.

PRACTICAL 2: Arithmetic Operations Using switch Case

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To perform addition, subtraction, multiplication and division using a switch statement.

Theory / Introduction

Operators are symbols used to perform operations on values. Arithmetic operators include +, -, * and /. A switch statement selects one branch from several alternatives based on the value of an expression.

The break statement prevents execution from falling through into the next case. The default branch handles an operation choice that does not match any listed case.

Important Points

- Use double when decimal values may occur.
- Check that the divisor is not zero before division.
- Each case label must be a constant value.

Program

```
class ArithmeticSwitch {
    public static void main(String[] args) {
        double a = 20, b = 5;
        char op = '*';

        switch (op) {
            case '+': System.out.println("Result = " + (a + b)); break;
            case '-': System.out.println("Result = " + (a - b)); break;
            case '*': System.out.println("Result = " + (a * b)); break;
            case '/':
                if (b != 0) System.out.println("Result = " + (a / b));
                else System.out.println("Division by zero is not allowed");
                break;
            default: System.out.println("Invalid operator");
        }
    }
}
```

Sample Output

Result = 100.0

Explanation

The operator variable contains '*'. The matching case multiplies 20 by 5 and displays 100.0.

Result

Thus, the program for arithmetic operations using switch case was written and its expected output was studied.

PRACTICAL 3: Find the Largest Element in an Array

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To find the largest value stored in a one-dimensional integer array.

Theory / Introduction

An array stores multiple values of the same data type under one variable name. Each element is accessed using an index, and Java array indexing begins at 0.

The array length property gives the number of elements. To find the largest value, assume the first element is the maximum and compare it with every remaining element.

Important Points

- Array size is fixed after creation.
- Valid indices range from 0 to length - 1.
- Initialize the maximum from the array rather than assuming it is positive.

Program

```
class LargestArray {
    public static void main(String[] args) {
        int[] numbers = {18, 42, 7, 31, 25};
        int largest = numbers[0];

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] > largest) {
                largest = numbers[i];
            }
        }
        System.out.println("Largest element = " + largest);
    }
}
```

Sample Output

```
Largest element = 42
```

Explanation

The loop compares each array element with largest. When a greater value is found, largest is updated.

Result

Thus, the program for find the largest element in an array was written and its expected output was studied.

PRACTICAL 4: Use Command-Line Arguments

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To accept values through command-line arguments and display them.

Theory / Introduction

Command-line arguments are values supplied after the class name when launching a Java program. Java receives them in the String array parameter of main(String[] args).

Each argument is initially a String. Numeric arguments must be converted using methods such as Integer.parseInt() before arithmetic can be performed.

Important Points

- Arguments are separated by spaces.
- args.length indicates the number of arguments.
- Compile using javac ClassName.java and run using java ClassName value1 value2.

Program

```
class CommandLineDemo {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("Usage: java CommandLineDemo <name> <age>");
            return;
        }
        String name = args[0];
        int age = Integer.parseInt(args[1]);

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

Sample Output

Command: java CommandLineDemo James 20
Name: James
Age: 20

Explanation

The first argument is read as a name and the second is converted to an integer. The program displays both values.

Result

Thus, the program for use command-line arguments was written and its expected output was studied.

PRACTICAL 5: Accept User Input Using Scanner

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To read user input from the keyboard using the Scanner class.

Theory / Introduction

Scanner is a class in the java.util package that can read input from sources such as the keyboard. An object is created with new Scanner(System.in).

nextLine() reads a complete line, next() reads a token, nextInt() reads an integer and nextDouble() reads a decimal number. Input must match the expected data type.

Important Points

- Import java.util.Scanner before using Scanner.
- Close the Scanner when input is no longer required.
- Mixing nextInt() and nextLine() may require consuming the pending newline.

Program

```
import java.util.Scanner;

class ScannerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Welcome " + name);
        System.out.println("Age: " + age);
        sc.close();
    }
}
```

Sample Output

```
Enter your name: James
Enter your age: 20
Welcome James
Age: 20
```

Explanation

Scanner reads a line for the name and an integer for age. The values are then printed.

Result

Thus, the program for accept user input using scanner was written and its expected output was studied.

PRACTICAL 6: Demonstrate Constructors and Methods

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To create a class with a parameterized constructor and an instance method.

Theory / Introduction

A class is a blueprint that defines data and behaviour. An object is an instance of a class. Fields store an object's state, while methods define operations that an object can perform.

A constructor initializes an object. It has the same name as the class and no return type. A parameterized constructor receives values when the object is created; a method can use those values to perform an operation.

Important Points

- The constructor runs when new creates an object.
- A constructor has no declared return type, not even void.
- Instance methods are called using an object reference.

Program

```
class Student {
    String name;
    int rollNo;

    Student(String n, int r) {
        name = n;
        rollNo = r;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }

    public static void main(String[] args) {
        Student s = new Student("James", 12);
        s.display();
    }
}
```

Sample Output

```
Name: James
Roll No: 12
```

Explanation

The constructor initializes name and rollNo. The display() method prints the object's data.

Result

Thus, the program for demonstrate constructors and methods was written and its expected output was studied.

PRACTICAL 7: Demonstrate Method Overloading

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To demonstrate compile-time polymorphism using overloaded methods.

Theory / Introduction

Method overloading means defining multiple methods with the same name in one class but with different parameter lists. The difference may be in the number, types or order of parameters.

The compiler determines which overloaded method to call from the arguments supplied. Changing only the return type does not create a valid overload.

Important Points

- Overloaded methods may have different return types if their parameter lists differ.
- Overloading is resolved at compile time.
- Choose parameter types that make the intended call unambiguous.

Program

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Sum of 2 integers = " + c.add(10, 20));
        System.out.println("Sum of 3 integers = " + c.add(10, 20, 30));
        System.out.println("Sum of decimals = " + c.add(2.5, 3.5));
    }
}
```

Sample Output

```
Sum of 2 integers = 30
Sum of 3 integers = 60
Sum of decimals = 6.0
```

Explanation

The compiler selects an add() method according to the number and types of arguments.

Result

Thus, the program for demonstrate method overloading was written and its expected output was studied.

PRACTICAL 8: Demonstrate Inheritance

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To implement single inheritance using the extends keyword.

Theory / Introduction

Inheritance is an object-oriented mechanism in which a subclass acquires accessible fields and methods from a superclass. It supports code reuse and represents an is-a relationship.

In Java, the extends keyword is used for class inheritance. A subclass can add its own methods and can use inherited public or protected members subject to access rules.

Important Points

- Java classes support single direct superclass inheritance.
- Constructors are not inherited.
- Private superclass members cannot be accessed directly by the subclass.

Program

```
class Person {
    void showName() {
        System.out.println("Name: James");
    }
}

class Student extends Person {
    void showClass() {
        System.out.println("Class: TYBCA");
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.showName();
        s.showClass();
    }
}
```

Sample Output

Name: James
Class: TYBCA

Explanation

Student extends Person and can call the inherited showName() method as well as its own showClass() method.

Result

Thus, the program for demonstrate inheritance was written and its expected output was studied.

PRACTICAL 9: Demonstrate Method Overriding

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To redefine a superclass method in a subclass.

Theory / Introduction

Method overriding occurs when a subclass provides its own implementation of an inherited instance method with a compatible signature. It allows the same method call to produce behaviour appropriate to the actual object.

Overriding is a basis of runtime polymorphism. The @Override annotation asks the compiler to verify that the method correctly overrides an inherited method.

Important Points

- The method name and parameter list must match.
- An overriding method cannot reduce the visibility of the inherited method.
- Static methods are hidden rather than overridden.

Program

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }

    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

Sample Output

```
Dog barks
```

Explanation

The reference type is Animal, but the actual object is Dog. At runtime, the overridden Dog.sound() method is invoked.

Result

Thus, the program for demonstrate method overriding was written and its expected output was studied.

PRACTICAL 10: Interface for Shape Area Calculation

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To define and implement an interface for calculating shape areas.

Theory / Introduction

An interface defines a contract that implementing classes agree to fulfil. It can declare abstract methods and may also contain default and static methods in modern Java.

A class uses the implements keyword to implement an interface and must provide implementations for its abstract methods unless the class is abstract. Interfaces help achieve abstraction and support multiple type inheritance.

Important Points

- Interface methods declared without a body are implicitly public and abstract.
- Implementing methods must be public.
- Use @Override to make implementations easier to verify.

Program

```
interface Shape {
    double area();
}

class Rectangle implements Shape {
    double length, width;

    Rectangle(double l, double w) {
        length = l;
        width = w;
    }

    public double area() {
        return length * width;
    }

    public static void main(String[] args) {
        Shape s = new Rectangle(5, 3);
        System.out.println("Area of rectangle = " + s.area());
    }
}
```

Sample Output

```
Area of rectangle = 15.0
```

Explanation

Rectangle implements Shape and defines area(). The interface reference calls the implemented method.

Result

Thus, the program for interface for shape area calculation was written and its expected output was studied.

PRACTICAL 11: String Manipulation Operations

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To demonstrate common String operations in Java.

Theory / Introduction

String represents a sequence of characters. String objects are immutable, which means operations such as concatenation produce a new String rather than changing the original object.

Methods such as length(), toUpperCase(), toLowerCase(), charAt(), substring() and equals() are commonly used to inspect or compare strings. Use equals() for content comparison rather than ==.

Important Points

- String indices begin at 0.
- substring(begin, end) includes begin but excludes end.
- String comparison is case-sensitive unless a case-insensitive method is used.

Program

```
class StringDemo {
    public static void main(String[] args) {
        String name = "James";
        System.out.println("Name: " + name);
        System.out.println("Length: " + name.length());
        System.out.println("Uppercase: " + name.toUpperCase());
        System.out.println("First character: " + name.charAt(0));
        System.out.println("Substring: " + name.substring(1, 4));
        System.out.println("Equals James: " + name.equals("James"));
    }
}
```

Sample Output

```
Name: James
Length: 5
Uppercase: JAMES
First character: J
Substring: ame
Equals James: true
```

Explanation

The program invokes String methods to find the length, change letter case, retrieve characters and compare content.

Result

Thus, the program for string manipulation operations was written and its expected output was studied.

PRACTICAL 12: Exception Handling Using try-catch-finally

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To handle an arithmetic exception using try, catch and finally blocks.

Theory / Introduction

An exception is an event that interrupts the normal flow of program execution. Exception handling allows a program to respond to certain runtime problems rather than terminating unexpectedly.

The try block contains code that may throw an exception. A catch block handles a matching exception, while finally normally runs whether or not an exception occurs and is commonly used for cleanup.

Important Points

- Catch a specific exception when possible.
- A catch block must follow its try block.
- finally is not an absolute guarantee under events such as forced JVM termination.

Program

```
class ExceptionDemo {
    public static void main(String[] args) {
        int a = 10, b = 0;

        try {
            int result = a / b;
            System.out.println("Result = " + result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Sample Output

```
Cannot divide by zero.
Finally block executed.
```

Explanation

Division by zero throws ArithmeticException. The catch block handles it and the finally block then executes.

Result

Thus, the program for exception handling using try-catch-finally was written and its expected output was studied.

PRACTICAL 13: Create a Simple Calculator Using Swing

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To create a graphical calculator window using Java Swing components and event handling.

Theory / Introduction

Swing is part of Java's GUI toolkit and provides platform-independent components such as JFrame, JLabel, JTextField and JButton. A JFrame represents a top-level application window.

Event-driven programming responds to user actions. An ActionListener can be attached to a button; its actionPerformed() method runs when the button is activated. Swing applications should create and update GUI components on the Event Dispatch Thread.

Important Points

- JFrame provides the main window.
- JTextField accepts text; parse it before numeric calculations.
- Use a layout manager instead of fixed coordinates for adaptable layouts.
- This example adds two numbers to keep the practical focused on GUI creation and event handling.

Program

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SwingCalculator extends JFrame implements ActionListener {
    JTextField first, second, result;
    JButton add;

    SwingCalculator() {
        setTitle("Java Swing Calculator");
        setLayout(new GridLayout(4, 2, 8, 8));

        add(new JLabel("First Number:"));
        first = new JTextField("12");
        add(first);

        add(new JLabel("Second Number:"));
        second = new JTextField("8");
        add(second);

        add(new JLabel("Result:"));
        result = new JTextField();
        result.setEditable(false);
        add(result);

        add = new JButton("Add");
        add.addActionListener(this);
        add(add);
```

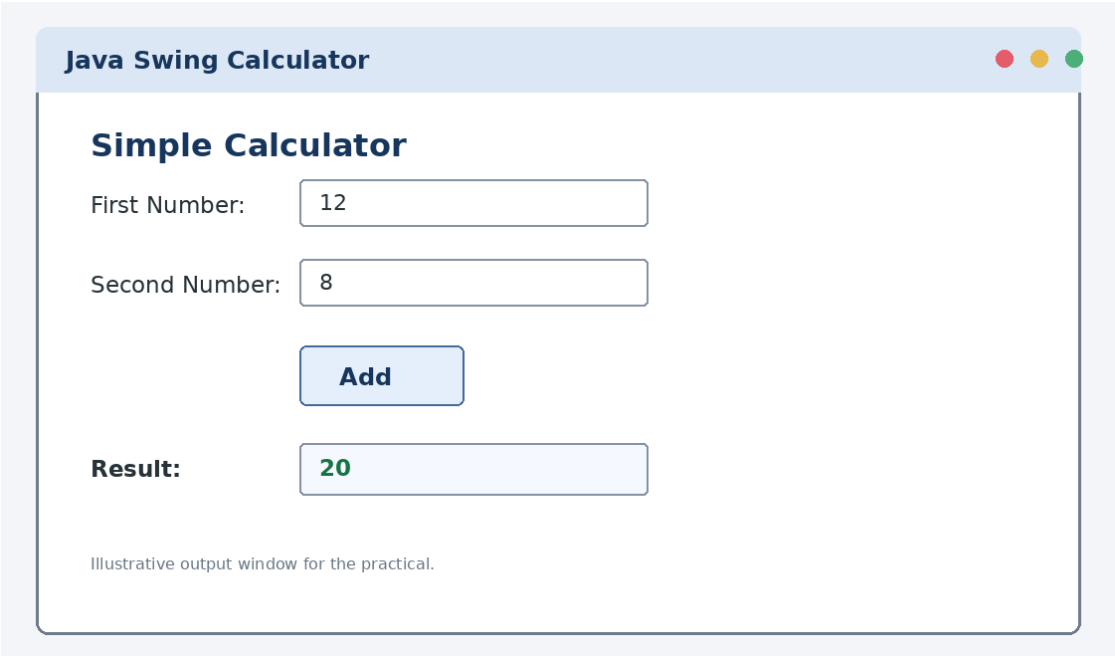
```
        setSize(380, 220);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            double a = Double.parseDouble(first.getText());
            double b = Double.parseDouble(second.getText());
            result.setText(String.valueOf(a + b));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Enter valid numbers.");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new SwingCalculator());
    }
}
```

Sample Output

A GUI window opens with First Number, Second Number and Result fields. Enter 12 and 8, then click Add; the Result field displays 20.0.



Illustrative GUI output window

Explanation

The constructor builds the GUI and registers the button listener. When Add is clicked, actionPerformed() reads both fields, converts them to numbers and displays their sum.

Result

Thus, the program for create a simple calculator using swing was written and its expected output was studied.

PRACTICAL 14: Create a Simple Login Form Using Swing

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To create a login form with username and password fields and display a result message.

Theory / Introduction

A login form is a GUI used to collect credentials. Swing provides JLabel for captions, JTextField for ordinary text, JPasswordField for password entry and JButton for the login action.

An ActionListener validates the entered values when the Login button is clicked. This practical uses fixed credentials only to demonstrate event handling; real systems must not store passwords in source code and require secure authentication.

Important Points

- Use JPasswordField rather than JTextField for password entry.
- Use getPassword() to retrieve a character array from JPasswordField.
- The hard-coded username and password below are for classroom demonstration only.

Program

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SwingLogin extends JFrame implements ActionListener {
    JTextField username;
    JPasswordField password;
    JButton login;

    SwingLogin() {
        setTitle("Java Swing Login Form");
        setLayout(new GridLayout(3, 2, 8, 8));

        add(new JLabel("Username:"));
        username = new JTextField();
        add(username);

        add(new JLabel("Password:"));
        password = new JPasswordField();
        add(password);

        login = new JButton("Login");
        login.addActionListener(this);
        add(login);

        setSize(360, 180);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setVisible(true);
    }
```

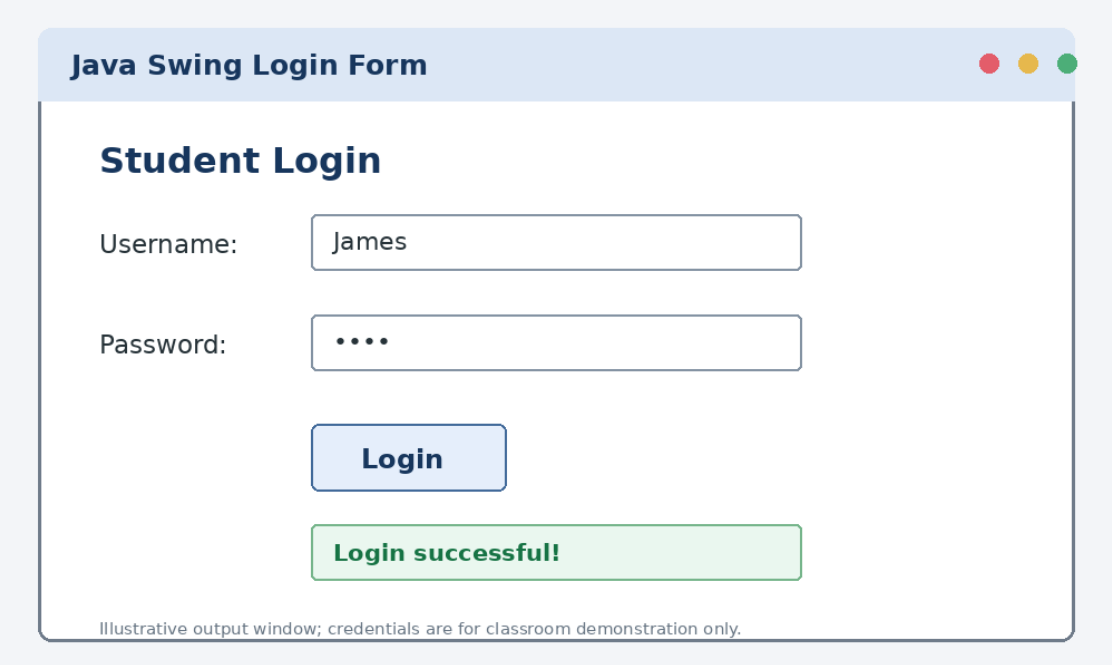
```
    public void actionPerformed(ActionEvent e) {
        String user = username.getText();
        String pass = new String(password.getPassword());

        if (user.equals("James") && pass.equals("1234")) {
            JOptionPane.showMessageDialog(this, "Login successful!");
        } else {
            JOptionPane.showMessageDialog(this, "Invalid username or password.");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new SwingLogin());
    }
}
```

Sample Output

The login window displays Username, Password and Login controls. For the classroom example, enter username James and password 1234; a “Login successful!” dialog appears.



Illustrative GUI output window

Explanation

The button event reads the username and password and compares them with the demonstration values. A message dialog reports success or failure.

Result

Thus, the program for create a simple login form using swing was written and its expected output was studied.

PRACTICAL 15: Demonstrate Database Connectivity Using JDBC

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To connect a Java application to MySQL using JDBC and retrieve student records.

Theory / Introduction

JDBC (Java Database Connectivity) is a Java API for communicating with relational databases. A JDBC driver translates JDBC calls into database-specific communication.

Typical JDBC steps are: load or register the driver when required, establish a Connection, create a Statement or PreparedStatement, execute a query, process the ResultSet and close resources. Modern JDBC drivers are generally registered automatically when present on the classpath.

Important Points

- Create the database and table before running the program.
- Add the MySQL Connector/J JAR to the classpath.
- Replace the example password with the password configured on your own MySQL server.
- Use try-with-resources to close JDBC resources.

Database Setup (run once in MySQL)

```
CREATE DATABASE tybcadb;
USE tybcadb;
CREATE TABLE studinfo (
   sid INT PRIMARY KEY,
   sname VARCHAR(50),
   cfees DOUBLE
);
INSERT INTO studinfo VALUES (1, 'James', 5000);
INSERT INTO studinfo VALUES (2, 'John', 6000);
```

Program

```
import java.sql.*;

public class JDBCSelect {
   public static void main(String[] args) {
      String url = "jdbc:mysql://localhost:3306/tybcadb";
      String user = "root";
      String password = "YOUR_MYSQL_PASSWORD";

      String sql = "SELECT sid, sname, cfees FROM studinfo";

      try (Connection con = DriverManager.getConnection(url, user, password);
           Statement st = con.createStatement();
           ResultSet rs = st.executeQuery(sql)) {

         while (rs.next()) {
            System.out.println(rs.getInt("sid") + " "
```

```
                + rs.getString("sname") + " "
                + rs.getDouble("cfees"));
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Sample Output

1 James 5000.0
2 John 6000.0

Execution Notes

- Download MySQL Connector/J and add its JAR file to the compile and runtime classpath.
- Windows classpath separator: semicolon (;). macOS/Linux classpath separator: colon (:).
- Example on Windows: javac -cp ".;mysql-connector-j-<version>.jar" JDBCSelect.java
- Example on Windows: java -cp ".;mysql-connector-j-<version>.jar" JDBCSelect
- Replace JDBCSelect with JDBCOperations for Practical 16 and replace YOUR_MYSQL_PASSWORD with your configured password.

Explanation

DriverManager creates a database connection. The SELECT query returns rows through ResultSet, and rs.next() moves through each row.

Result

Thus, the program for demonstrate database connectivity using jdbc was written and its expected output was studied.

PRACTICAL 16: Insert, Update and Delete Database Records

Course Code: CA-315 | Practical based on JAVA Programming

Student Name: _____ Roll No.: _____

Aim: To perform INSERT, UPDATE and DELETE operations on MySQL records using JDBC PreparedStatement.

Theory / Introduction

SQL data manipulation statements modify table data. INSERT adds a new row, UPDATE changes existing row values and DELETE removes rows that satisfy a condition.

PreparedStatement represents a parameterized SQL statement. Placeholders (?) are filled using setter methods such as setInt(), setString() and setDouble(). Parameterized statements improve clarity and help prevent SQL injection when values are supplied as parameters.

Important Points

- Run INSERT before UPDATE and DELETE for the same sample record.
- Always use a suitable WHERE clause for UPDATE and DELETE.
- executeUpdate() returns the number of rows affected.
- Use a transaction when several changes must succeed or fail together.

Program

```
import java.sql.*;

public class JDBCOperations {
   public static void main(String[] args) {
      String url = "jdbc:mysql://localhost:3306/tybcadb";
      String user = "root";
      String password = "YOUR_MYSQL_PASSWORD";

      try (Connection con = DriverManager.getConnection(url, user, password)) {
         String insertSQL = "INSERT INTO studinfo VALUES (?, ?, ?)";
         try (PreparedStatement ps = con.prepareStatement(insertSQL)) {
            ps.setInt(1, 3);
            ps.setString(2, "Robin");
            ps.setDouble(3, 7000);
            System.out.println("Inserted rows: " + ps.executeUpdate());
         }

         String updateSQL = "UPDATE studinfo SET cfees = ? WHERE sid = ?";
         try (PreparedStatement ps = con.prepareStatement(updateSQL)) {
            ps.setDouble(1, 7500);
            ps.setInt(2, 3);
            System.out.println("Updated rows: " + ps.executeUpdate());
         }

         String deleteSQL = "DELETE FROM studinfo WHERE sid = ?";
         try (PreparedStatement ps = con.prepareStatement(deleteSQL)) {
            ps.setInt(1, 3);
            System.out.println("Deleted rows: " + ps.executeUpdate());
         }
```

```
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Sample Output

```
Inserted rows: 1
Updated rows: 1
Deleted rows: 1
```

Execution Notes

- Download MySQL Connector/J and add its JAR file to the compile and runtime classpath.
- Windows classpath separator: semicolon (;). macOS/Linux classpath separator: colon (:).
- Example on Windows: javac -cp ".;mysql-connector-j-<version>.jar" JDBCSelect.java
- Example on Windows: java -cp ".;mysql-connector-j-<version>.jar" JDBCSelect
- Replace JDBCSelect with JDBCOperations for Practical 16 and replace YOUR_MYSQL_PASSWORD with your configured password.

Explanation

The program inserts Robin, updates the fee for student ID 3 and deletes that same record. Each executeUpdate() reports the number of affected rows.

Result

Thus, the program for insert, update and delete database records was written and its expected output was studied.

Practical Submission Checklist

- Write the practical number and title clearly in the record.
- Include the aim, theory, program, sample output, explanation and result.
- Verify the program by compiling and executing it.
- For Swing practicals, attach or draw the observed application window and record the user action performed.
- For JDBC practicals, mention the database name, table structure and any connection configuration used.